SPRINT 5 · MATEMÁTICAS Y ESTADÍSTICA EN CÓDIGO · S05.2

Descenso del gradiente y regresión desde cero

Así aprende un modelo: mide su error, mira hacia dónde baja y da un paso.
Lo programamos con NumPy, sin librerías de IA.

La derivada: la pendiente en un punto

- Positiva: la curva sube. Negativa: baja. Cero: estamos en un mínimo o un máximo.
- Se aproxima sin fórmulas con un `h` muy pequeño:

```
def derivada_numerica(func, x, h=1e-6):          # Ejercicio 1
    return (func(x + h) - func(x - h)) / (2 * h)
```

Función de coste: el error cuadrático medio

```
def mse(y_real, y_pred):                          # Ejercicio 2
    return np.mean((y_real - y_pred) ** 2)

mse_ingenuo = mse(ocupacion, ocupacion.mean())   # predecir siempre la media
```

- Al cuadrado: los errores grandes pesan más y no se cancelan.
- El modelo «ingenuo» (siempre la media) es **la referencia a batir**.

Descenso del gradiente en una dimensión

Empieza en cualquier sitio y da pasos cuesta abajo: **`x ← x − tasa · pendiente`**

```
# g(x) = x² - 6x + 20, mínimo en x = 3, derivada 2x - 6
x = 10
for _ in range(50):                                  # Ejercicio 3
    x = x - 0.1 * (2 * x - 6)
```

La **tasa de aprendizaje** es el tamaño del paso: muy pequeña = lento; muy grande = se pasa del mínimo y puede divergir.

Regresión lineal: modelo y gradiente

- Modelo: `ocupación ≈ w0 + w1 · temperatura`, es decir `y_pred = X @ w`.
- Gradiente del MSE: `(2 / n) · Xᵀ (X w − y)`.
- **Estandarizamos** la variable (media 0, desv. 1) para que el descenso sea estable.

```
def gradiente_mse(X, y, w):                          # Ejercicio 4
    return (2 / len(y)) * X.T @ (X @ w - y)
```

El bucle de entrenamiento

```
def entrenar(X, y, tasa=0.1, epocas=200):          # Ejercicio 5
    w = np.zeros(X.shape[1])
    historial = []
    for _ in range(epocas):
        w = w - tasa * gradiente_mse(X, y, w)
        historial.append(mse(y, X @ w))
    return w, historial
```

El `assert` compara tu gradiente con el calculado por derivadas numéricas. La curva `historial` debe **bajar**.

Elegir la tasa de aprendizaje

Síntoma	Causa	Arreglo
El error sube o da `nan`	Tasa demasiado grande	Dividirla entre 10
Baja, pero muy despacio	Tasa pequeña o variables sin estandarizar	Estandarizar; más épocas
Pendientes absurdas en unidades originales	Olvidar deshacer la estandarización	`coef / desviación`

Ejercicio 6: con `tasa = 2.5` y 30 épocas, comprueba que el entrenamiento **diverge**.

Comprobar con scikit-learn

```
pendiente = w[1] / desv                          # de «por desv. típica» a «por grado»
intercepto = w[0] - w[1] * media / desv
coinciden = np.isclose(pendiente, lr.coef_[0], atol=1e-3)   # Ejercicio 7
```

- Si tu modelo «desde cero» coincide con `LinearRegression`, lo has entendido.
- Mini-reto: regresión **múltiple** (temperatura, festivo y precio medio) con la misma función `entrenar`.
- Todo lo que viene después (redes neuronales incluidas) es este mismo bucle con más pesos.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S05_03_gradiente_regresion — 70 min

Descenso del gradiente y regresión desde cero

sprints/S05/notebooks/S05_03_gradiente_regresion.ipynb